

MILESTONE 3

What is RAG?

RAG, or Retrieval-Augmented Generation, is a cutting-edge artificial intelligence technique that enhances the capabilities of large language models (LLMs) by connecting them to external knowledge sources. In essence, it allows these models to "look up" information before answering a question, leading to more accurate, up-to-date, and contextually relevant responses.

Embedding Models:

- **E5 Family (Embeddings from Instructions):**

E5 (from Microsoft & Intfloat) is based on **instruction-tuned contrastive learning**.

Instead of only learning from sentence pairs, it learns from **instruction–response pairs**

Main goal - To generate embeddings optimized for **retrieval and ranking** tasks. Unlike plain similarity models (like SBERT), E5 is **instruction-tuned**, meaning it understands “what is being asked” (query intent) and “what content can answer it” (passage meaning). This makes it ideal for **RAG pipelines** where user queries need to match relevant document chunks.

Trained with - massive datasets from search engines, QA pairs, and web text, with explicit “query/passage” formatting.

Strengths -

- Open-source and can be used locally with good performance on commodity GPUs
- Handles diverse natural language phrasing, so it performs well even on reworded or indirect questions.
- Designed specifically for **retrieval-based tasks** like RAG — excels at mapping questions to the right document sections.

- Instruction-tuning makes it **query-aware**, improving precision in answer retrieval.

Limitations -

- Slightly heavier (~768 dimensions).
- Requires proper **prefix formatting** (“query:” / “passage:”) for optimal results.
- Only 512 tokens

- **BGE Family (BAAI General Embedding)**

Similar to E5 but tuned for **dense retrieval** efficiency.

Main goal - To produce embeddings that are highly effective for **dense retrieval and semantic search**, while being efficient enough for large-scale RAG systems. It’s optimized for **retrieval robustness** and maintains **semantic consistency** even when model size is small.

Trained with Large multi-domain dataset including natural text, Q/A pairs, web docs, and multilingual sources.

Strengths -

- Specifically tuned for **dense retrieval in RAG**, where matching query and context meaning matters more than syntax.
- Delivers **strong performance in multilingual and domain-agnostic document search**, making it versatile for global or diverse datasets.
- Smaller models (like bge-small-en or bge-base-en) give **excellent performance-to-size ratio**, ideal for local or edge RAG setups.
- Strong alignment with LLM-based question answering pipelines — integrates well with vector databases (like FAISS, Milvus, Chroma).

Limitations -

- Has slightly **less fine-grained contextual nuance** for long, descriptive paragraphs compared to larger OpenAI models.

- Performance may drop if text is very abstract or literary (e.g., philosophical or emotional context).

- **OpenAI Embedding Models (text-embedding-3-small / large):**

Main goal - To provide **universal, domain-general embeddings** for text, code, and documents — designed to capture meaning, context, and relationships across wide-ranging corpora. Perfect for **production-grade RAG** where precision and scale are critical.

Trained with Massive multi-domain corpus: web text, documentation, code, Wikipedia, and instruction data. Models learn not only meaning but also *relations* and *metadata semantics*.

Strengths

- State-of-the-art semantic accuracy.
- Handles long context windows.
- Excellent for production-scale similarity and clustering.

Limitations -

- Cloud only (requires API).
- Cost per token (though small models are very cheap).
- No local interpretability or open weights.

- **FashionSigLIP:**

FashionSigLIP is a domain-tuned version of Google's Sigmoid Loss CLIP architecture, optimized for fashion and e-commerce imagery.

It retains CLIP's multimodal nature but is specifically trained to handle fine-grained style, texture, and material distinctions common in apparel data.

Main goal - To generate high-fidelity multimodal embeddings that capture **color, cut, fabric, and aesthetic “vibe”** of fashion items for recommendation, similarity search, and retrieval.(both image and text - multimodal)

Trained with Specialized **fashion e-commerce datasets** containing product listings, outfit metadata, and descriptive text from online catalogs.

Strengths

- **Fine-grained style sensitivity** — can tell apart fabrics, necklines, and patterns within similar categories.
- Excellent **cross-modal alignment** for both image→text and text→image retrieval.
- **Lightweight and fast**, making it ideal for embedding large fashion inventories.

Limitations –

- Slightly narrower domain — not suited for generic images (like landscapes or food).
- Requires high-quality fashion metadata for best results.

- **DINOv2 (Meta AI)**

DINOv2 (Self-Distillation with No Labels) from Meta AI is a **pure vision transformer**, trained via self-supervised learning.

Unlike CLIP-style models, DINOv2 doesn't use text — it learns powerful visual representations directly from unlabeled image data.

Main goal - To produce robust visual embeddings that capture **style, structure, and visual patterns** without needing captions or annotations.

Trained with - Billions of images from diverse visual domains, using a **self-distillation** process that forces the network to learn meaningful image clusters and invariances.

Strengths

- **Exceptional visual embedding quality** for clustering, similarity, and unsupervised grouping.
- Handles fine-grained visual differences (textures, shapes, symmetry).
- **Very efficient** and flexible for downstream visual tasks.

Limitations –

- No text encoder — cannot perform text→image retrieval (“red dress” won’t work directly).
- Requires separate text model or cross-modal alignment for multimodal tasks.

- **FashionCLIP (by Marqo)**

FashionCLIP is a domain-specialized adaptation of CLIP trained exclusively on **fashion datasets**, focusing on apparel, accessories, and footwear.

It aligns fashion product descriptions with corresponding catalog images to learn embeddings that understand **style, silhouette, and trend**.

Main goal - To provide fashion-aware multimodal embeddings that accurately represent both the **visual look** and **descriptive language** of clothing and accessories.

Trained with - Large-scale **fashion industry data** (images + descriptions) curated from product catalogs and style-focused datasets.

Strengths

- Excellent understanding of **fashion attributes** like silhouette, material, and seasonality.

- Text and image embeddings are **well-aligned** for descriptive queries.
- Designed for fashion-specific retrieval, style search, and recommendation systems.

Limitations –

- Smaller training set compared to SigLIP — slightly weaker **cross-domain generalization**.
- **Slower inference** for very large catalogs.

Preferred embedding model:

1. Part A – **BGE family** - because they are specifically optimized for dense retrieval in RAG, offering a strong balance between accuracy, efficiency, and local usability. They provide high-quality semantic matching while remaining lightweight enough for scalable, on-device retrieval.
2. Part B – **FashionSigLIP** - because it offers the best balance of fine-grained fashion understanding and text–image alignment, unlike FashionCLIP which has limited generalization and DINOv2 which lacks text comprehension.

Re-embedding:

- When changes are made in the embedding doc we will simply delete all the embeddings generated for the doc from the vector database (delete the collection) and re-embed the doc again.
- With respect to the products, if a certain product is removed from the catalog, we will just delete the corresponding embedding.
- If any product's info is edited, we delete the embedding of that product and re-embed it.

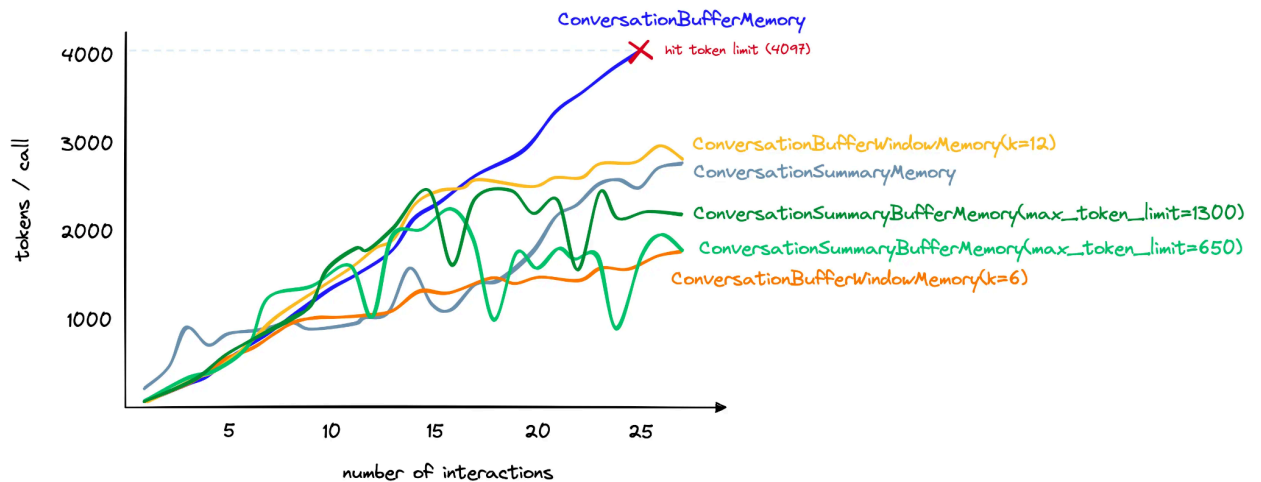
How to maintain chat history and feed to LLM?

Conversational history : For implementing this task in our chatbot, a conversational memory will be maintained, enabling a coherent conversation.

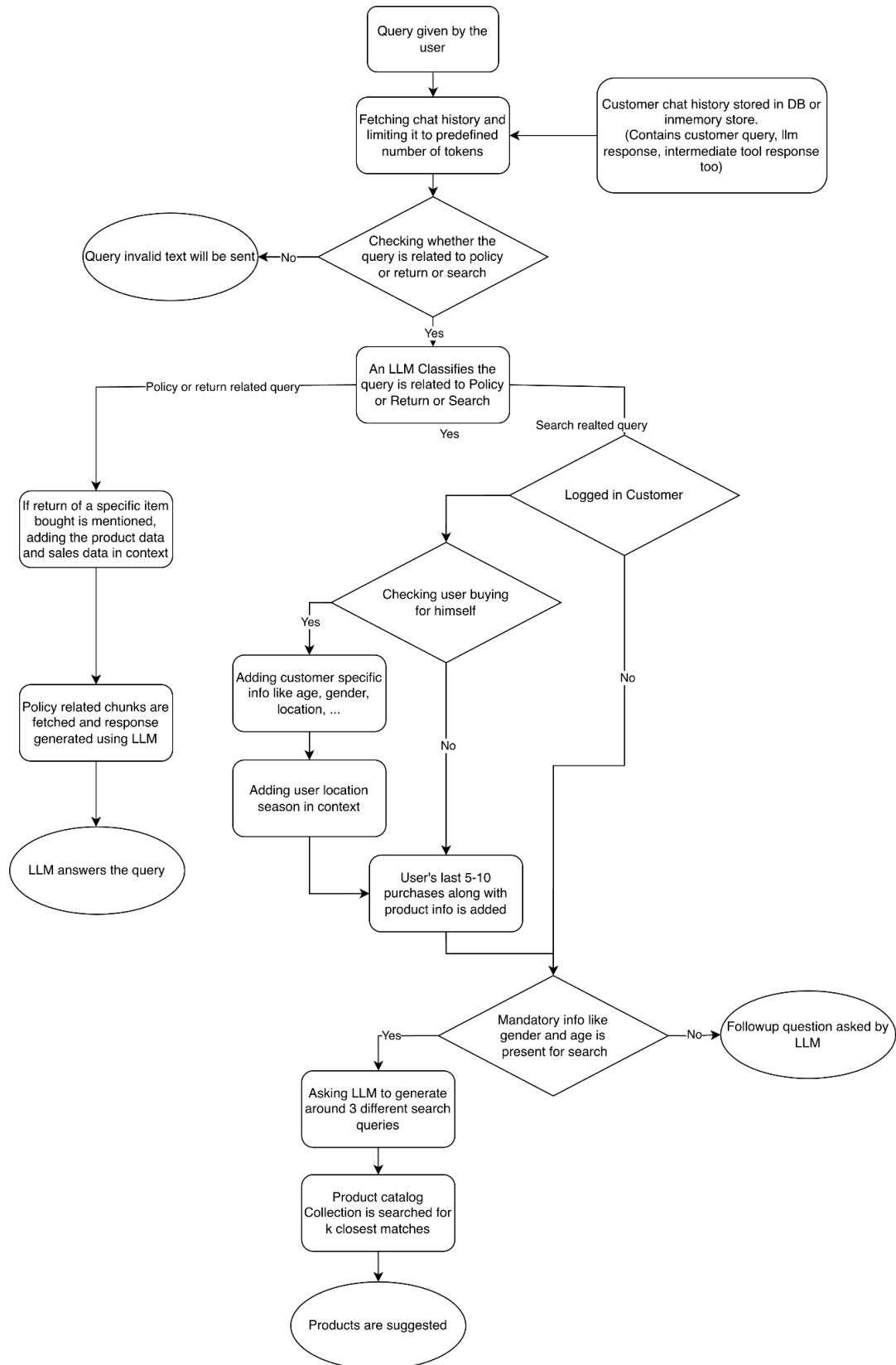
Breakdown of the pipeline-

- Initialize the *ConversationChain* with a LLM model.
- We will be keeping the conversational memory in one of the parameters for the model in *{history}*, and
- For saving each customer's chat history from a session to a persistence storage, we will be using Redis (planning to save latest session's chat history)
- Since maintaining long chat histories of customers could lead to exceeding the maximum token limit, we will be implementing conversation summarizing techniques.

The difference between such techniques (e.g. *ConversationSummaryMemory*) and without it (*ConversationBufferMemory*) is shown in the chart below.



Query Flow:



Flow Explanation:

Checking whether the query is related to policy or return or search:

LLM will be fed the user query along with history. It will also be fed an instruction that it is a chatbot which will answer about policies of an online clothing store or it will help customers search for clothes. If the query along with the history is related to the tasks provided to the LLM, we will move further else the LLM will ask the customer to ask relevant questions.

An LLM Classifies the query is related to Policy or Return or Search:

- Here LLM is instructed to classify the query into one of the three tasks. The query asks for any info about store policy or any return instruction or it is a search query for clothes. Based on the classification provided by the LLM we will choose a path and move further.
- If the query is a simple policy related query, we will add context via embeddings retriever.
- If the query is related to returning a product, then the product is searched in the product catalog and also the sales data is added to the context along with the context we will be getting from searching the policy doc via embeddings retriever.
- For product search related queries lots of other context needs to be added which will be explained in the following sections.

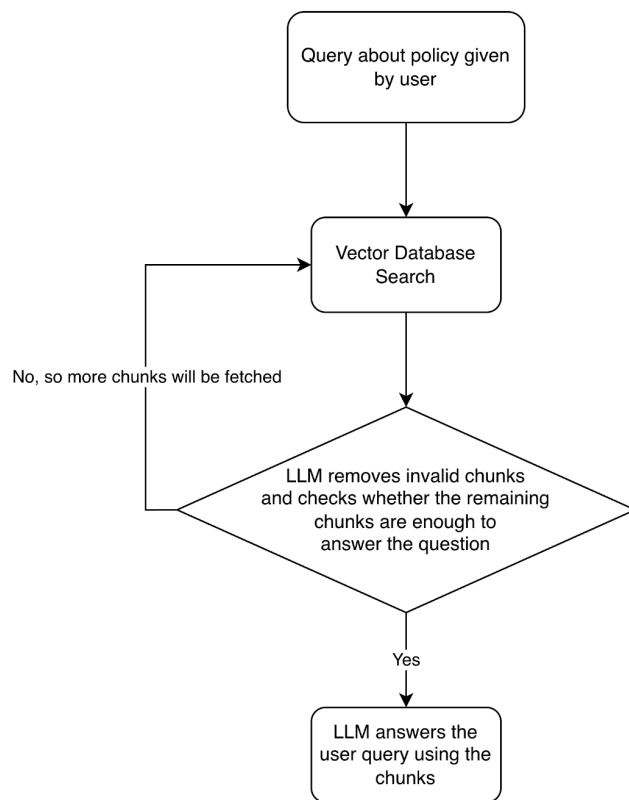
Context building for product search:

- If the customer is a logged in user, then we can add his/her data like age and gender in context if he is buying for himself.
- If the customer is a logged in user and not buying for himself, we will try to get age and gender from the query, if not possible, the LLM will ask a follow-up question. The same approach will be used in case of guest users.
- If we are able to get location data (can get it from user DB), an api call to fetch the season of a particular zipcode can be used. This season info can be added into context.
- For logged in customers we also have previous sales data. Using that data we can add into context, at what price point the customer purchases products, how much discount was there on the products, category, sub category and style of products purchased, etc.

- If from the context built if all the mandatory info is present, we will ask LLM to come up with a search query using the context and search for relevant products in Vector DB. If mandatory info is not available in context then a followup question will be asked by the LLM.

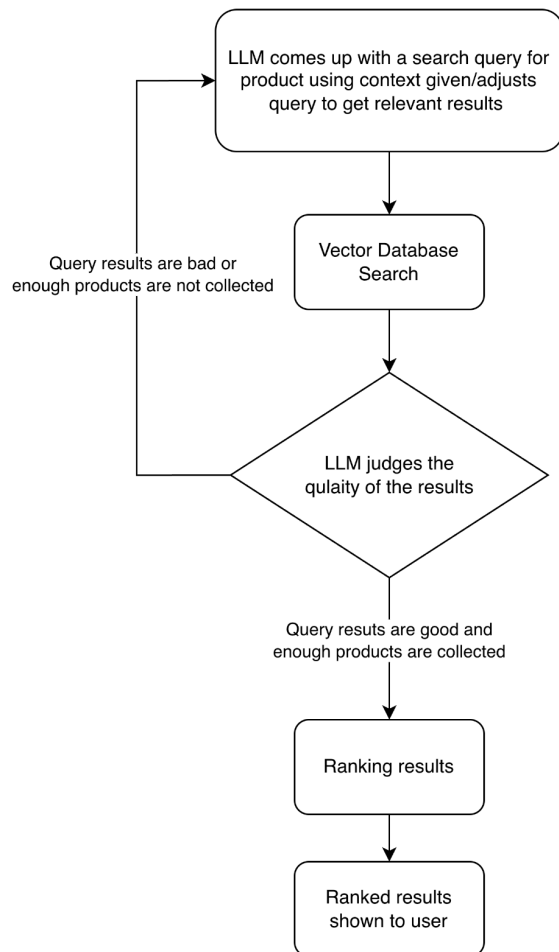
Search in Vector DB elaborated:

Policy Doc Search:



- We will search for lets say 5 chunks from policy doc to answer user query.
- The LLM will check whether the chunks are valid and remove invalid chunks.
- Also the LLM will check whether there is enough context to answer the user query. If not then more chunks will be fetched to answer the query (We can cap the total number of search by 3)

Product Catalog Search:



- The LLM will give out a search query and ask for matching products to the VDB.
- Then judging will be done whether the products given matches the search query or not (by an LLM).
- If not enough products (say 5) are relevant, then LLM will be made to make adjustments in search query and the VDB search will be triggered again based on the irrelevant products got (using the irrelevant product fetched, the LLM can identify what was the mistake in the search query and try to rectify it).
- This back and forth operation will be tried 2-3 times after which the products deemed irrelevant will be shown (Most probably because the product user is searching for is not available in the catalog).

- After we have the products which we want to show the customer, we will rerank that using the product info and context built. (this determines the order in which we need to show the search results)
- Currently we think we can use an LLM to do it. But we need further analysis in this part and will decide the approach in future.
- Currently we are thinking of a strategy, in which we embed the text columns of Product catalog and images separately and concatenate them.

Embedding only the Text Columns:

<https://colab.research.google.com/drive/1vLbQXN8NOBWynEf4U16hXTec5v9bhosz?authuser=1#scrollTo=416491f2-c891-4763-ab10-67d7a2105915>

Embedding the images using Clip:

<https://drive.google.com/file/d/12PrSB1A-Su0MROaUSxf1ZsIsz66Qdne1/view?usp=sharing>